

第十九届黑龙江省大学生程序设计竞赛 - 正式赛

开始时间	05/12/2024 9:00:00 AM	结束时间	05/12/2024 2:00:00 PM	答题时长	300分钟
答卷类型	空白答卷	试卷总分	3600		

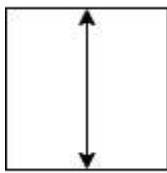
编程题 3600 分

A Wor is a plumbing repairman. 300分

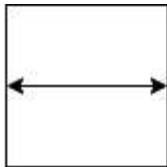
Wor often helps neighbors in the community to repair broken plumbing. With a high talent for fixing, Wor is confident in his capability. However, now he is facing trouble, as a serious accident happened in the central water supply station that interrupted the whole city's water supply. Because of the complexity of the pipe network, he failed and felt sad. Now, he is seeking your help. Can you help him?

There are four types of plumbing in the central water supply station. Now we define the operation $'$, which means a 90 degree clockwise rotation of the water pipe.

- *type 1 plumbing*: I -type water pipe and $-$ -type water pipe.
When we do the operation I' , we can get the $-$ -type pipe. Meanwhile, operation $-'$ can get the I -type pipe.
 - I -type water pipe represents the vertical water pipe, which means water flow can pass from bottom to top or from top to bottom.

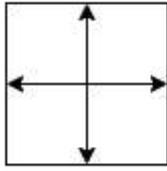


- $-$ -type water pipe represents the horizontal water pipe, which means water flow can pass from left to right or from right to left.



- *type 2 plumbing*: $+$ -type water pipe.
No matter how we use the operation $+$ ', we can only get the $+$ -type pipe.
 - $+$ -type water pipe represents the crossing water pipe, which means water flow can only pass from left to right, right to left, bottom to top, or

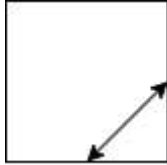
from top to bottom.



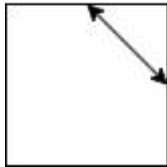
- *type 3 plumbing*: p -type water pipe, b -type water pipe, d -type water pipe, and q -type water pipe.

Operation p' would get the q -type water pipe, q' would get the d -type water pipe, d' would get the b -type water pipe, and b' would get the p -type pipe.

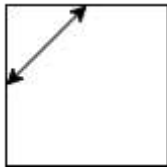
- p -type water pipe represents the left-top corner pipe, which means water flow can pass from bottom to right or from right to bottom.



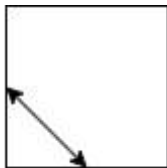
- b -type water pipe represents the left-bottom corner pipe, which means water flow can pass from top to right or from right to top.



- d -type water pipe represents the right-bottom corner pipe, which means water flow can pass from top to left or from left to top.



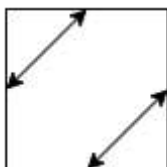
- q -type water pipe represents the right-top corner pipe, which means water flow can pass from bottom to left or from left to bottom.



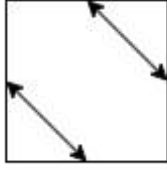
- *type 4 plumbing*: x -type water pipe and y -type water pipe.

Doing operation x' , we can get the y -type water pipe, and we can get the x -type pipe when we do the operation y' .

- x -type water pipe represents the combination of d -type water pipe and p -type water pipe, looks like dp -type water type, which means water flow can pass from bottom to right, right to bottom, top to left, or from left to top.



- *y-type* water pipe represents the combination of *q-type* water pipe and *b-type* water pipe, looks like *qb-type* water type, which means water flow can pass from bottom to left, left to bottom, top to right, or from right to top.



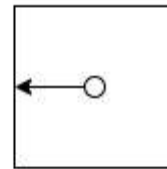
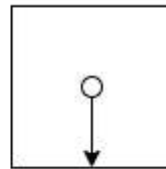
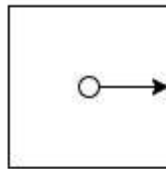
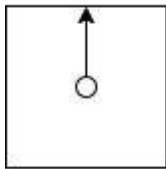
- *type 5 plumbing*: *starting tube* and *ending tube*, which is the entrance of the water flow and the exit of the water flow. There is only one starting tube and one ending tube.

Starting tube and ending tube have only one direction, *u, o, l, r*, respectively representing the water flow out and in direction as up, down, left, or right.

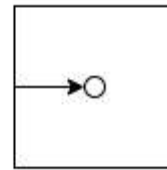
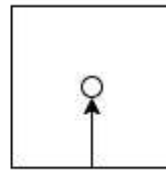
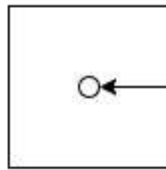
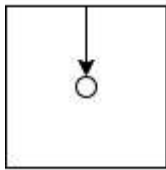
Attention please, starting tube and ending tube cannot rotate.

The order of following pictures stick to *u, r, o, l*.

- *starting tube*:



- *ending tube*:



- In fact, distinguishing between starting tube and ending tube is meaningless, so there is no distinction between starting tube and ending tube in the data.

Now, Wor would give you the initial status of the plumbing matrix, which is an $n \times m$ matrix $a_{i,j}$. Your mission is to find a final fixed matrix $b_{i,j}$ that is available by only using the operation $'$ in the initial plumbing matrix, and the water flow can pass through from the starting tube to the ending tube.

Input

The first line contains two integers n, m ($2 \leq n, m \leq 50$) representing the size of the matrix as $n \times m$.

Then following n lines and m columns character matrix represent the initial status of the plumbing matrix $a_{i,j}$ ($a_{i,j} \in \{i, +, p, x, u, o, l, r\}$).

The character *u, o, l, r* represents the starting pipe or ending pipe direction. There must be two characters that are *u, o, l, r*, no matter which is the starting pipe or ending pipe.

We ensure that the total number of p -type pipes and x -type pipes is less than or equal to 20.

Output

An n lines and m columns character matrix represents the fixed status of the plumbing matrix $b_{i,j} (a_{i,j} \in \{i, -, +, p, b, d, q, x, y, u, o, l, r\})$ that can be obtained by only using the operation $'$ in the initial matrix, and the water flow can pass through from the starting tube to the ending tube.

Example1

input

```
5 7
pppiipi
iiupixl
ipi+pii
pipppii
iipiipi
```

output

```
pqp--qi
iiup-xl
ib--+qii
b-qbdii
iib--di
```

Example2

input

```
2 5
riiil
iiiiii
```

output

```
r---l
iiiiii
```

Example3

input

```
3 3
oii
pip
iiu
```

output

```
oii
b-q
iiu
```

Example4

input

```
5 2
oX
+X
+X
+X
uX
```

output

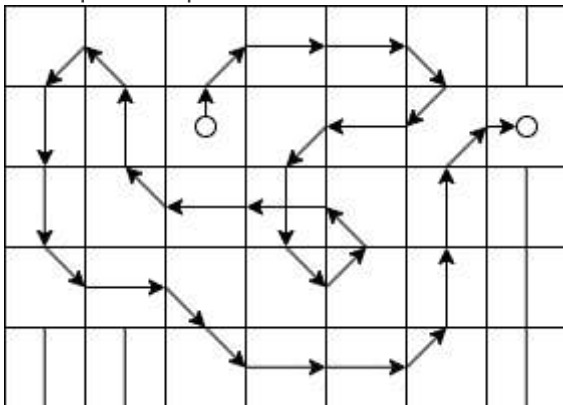
```
oX
+X
+X
+X
uX
```

Note

There may be many legal solutions, you only need to output one of them.

The data ensures that there is a legitimate solution, so you don't need to consider situations where there is no solution.

Example 1 explanation:



- B Snow has a string, and now he wants to use magic to shorten this string. Each time he casts a spell, he can eliminate three adjacent identical characters. But Snow finds it too time-consuming to cast the spell repeatedly, so he hopes you can help him calculate what the shortest form of the string will be after using magic any number of times.

300分

Input

Input a string, ensuring that it only contains lowercase letters, and the length of the string does not exceed 10^6 .

Output

Output a string representing the shortest string after casting magic.If the entire string is eliminated, the output should be "NAN".

Example1

input

```
bcabaaabbccabcc
```

output

```
bcaccabcc
```

Example2

input

```
cccpppccc
```

output

```
NAN
```

- C There is a rich kingdom on the mainland, and the road structure of the whole mainland can be regarded as a rooted tree. The capital is located at node 1 as the root of the whole tree.

300分

In the past, people lived in peace and contentment. However, with the evil monsters flooding the mainland, people are suffering. The king decided to send the army to hunt the monsters. Before that, the king sent out reconnaissance teams, to watch for monster traces, if found at a certain node, it meant that there was at least one monster in the subtree. After a period of searching, the reconnaissance team found some nodes with traces of the monster, we call the nodes with traces of the monster as markers.

However, the reconnaissance is still not over, and the reconnaissance team may make new discoveries, which may be the discovery of new markers, or it may be determined that the previous markers are false positives, that is, cancel the marking of a certain node. And you, as the brain of the Kingdom, your task is to update how many monsters at least when new discoveries emerge.

Input

The first line of input contains 3 integers n, m, q , indicating the number of nodes of the tree, the number of markers at the beginning, and the number of new discoveries.

The next line contains $n - 1$ integers $f_2, f_3 \cdots f_n$ indicating the father of nodes.

The next line contains m integers $a_1, a_2 \cdots a_m (a_i \leq n)$ indicating the nodes with markers at the beginning. Guarantee that if $i \neq j, a_i \neq a_j$.

Each of the following q lines contains 2 integers $opt, x (x \leq n)$. $opt = 1$, which mean marked node x ; $opt = 2$, which mean cancel the marking of node x .

Guarantee that if $opt = 1$, node x is not marked before it, if $opt = 2$, node x is marked before it.

It guaranteed that $1 \leq n, m, q \leq 1 \times 10^6$.

Output

Output $q + 1$ lines, each line contains an integer. The first line indicates the minimum number of monsters possible before new discoveries emerge. The $i + 1$ line indicates the minimum number of monsters possible after the $i - th$ new discoveries emerge.

Example

input

```
8 3 4
1 1 2 2 3 3 4
2 6 8
1 3
1 7
2 3
2 6
```

output

```
2
2
3
3
2
```

Note

The problem requires a large amount of input, please use the appropriate input method.

- D A and B are playing card games, the game starts with A and B each having some health, called hpa and hpb . Each player starts with n cards, called $a_1, a_2 \dots a_n$ and $b_1, b_2 \dots b_n$, and each player plays one card he has not played before per turn. There are two types of cards: attack cards and defense cards. Each attack card has an atk , if one of them plays an attack card and the other doesn't play a defense card, the other's health will reduce the atk of the attack card; If the other player plays a defense card, his health doesn't decrease. If a player plays a defense card, it has no effect other than to defend against the other's attack.

When a turn ends, if any player's health is less than or equal to zero, the game ends. If only one player's health is less than or equal to zero, the other wins. If both player's health is less than or equal to zero, it is a draw. If after all turns, no player has zero or less health, it is also a draw.

We call the sequence of A's cards played $p_1, p_2 \dots p_n$, and call the sequence of B's cards played $q_1, q_2 \dots q_n$, they are permutations from 1 to n . If the game hasn't been over before the $i - th$ turn, in the $i - th$ turn, A will play a_{p_i} , and B will play b_{q_i} .

Now you know A and B's cards. The question is whether there exists a pair of sequences of A and B's cards played that will make A win.

Input

The first line contains an integer T , indicating the number of test sets.

And then for each set of data. The first line contains three integers n , hpa , and hpb , which indicate the number of cards per player and the initial health of two players. The second line contains n integers $a_1, a_2 \dots a_n$, indicating A's cards, where $a_i = -1$ indicates the defense card, and in other cases indicates the attack card and a_i is the atk . The third line contains n integers $b_1, b_2 \dots b_n$, indicating B's cards, in the same way.

It guaranteed that $T \leq 100$. For all T test sets, the sum of n is less than 10^5 . $1 \leq hpa, hpb \leq 10^8$. $1 \leq a_i, b_i \leq 10^8$ or equal to -1 .

Output

For each set of data, output a string, "yes" or "no", indicating whether there exists a pair of sequences of A and B's cards played that will make A win.

Example

input

```
2
3 4 9
-1 7 3
```



```
5 2 1
5 9 11
3 -1 6 4 1
-1 -1 10 5 2
```

output

```
yes
no
```

E Joey and Grey are playing a poker game.

300分

Initially, Joey has n poker cards and Grey has none. They would repeat the following steps to help Grey get as many cards as possible:

1. Randomly pick a card from the card pile. This card is called **FATE**. Since the pile is very large and contains much more cards than one pack, the probability for one to pick/get cards of hearts, spades, diamonds and clubs will always be a , b , c , d respectively.
2. Joey can (not must) use one card he owned to exchange the **FATE**, that means the **FATE** will be changed by the one Joey uses (Joey will lose this one) and Joey will get the old **FATE** card in step 1. If Joey uses one card of spade in this process, he will get another new card randomly from the pile.
3. If now the **FATE** is spade or club, Grey can get this card and start another round from step 1, otherwise they stop repeating and begin to count how many cards Grey got.

They are curious about one question --- if they operate optimally, how many cards could Grey get expected.

Input

The first line of input contains four integers, they are $100a$, $100b$, $100c$, $100d$. We guarantee that $0 \leq 100a, 100b, 100c, 100d \leq 100$ and $100a + 100b + 100c + 100d = 100$.

The second line of input is a string of length n to represent the cards Joey owns originally, in which H represents hearts, S for spades, D for diamonds and C for clubs. We promise $1 \leq n \leq 100$.

Output

Just one integer represented the maximum expected number of cards modulo 998244353 Grey can get. If this number is infinity, please output *INF*.

Example

input

```
25 25 25 25
S
```

output

```
6
```

Example2

input

```
0 100 0 0
HSDC
```

output

```
INF
```

Note

For the first example, the maximum expected number of cards Grey can get is 6. I can not provide all the cases here (which are infinity), but one possible case that Grey gets 6 cards is that:

The first round, they picked one heart. Joey used his spade to exchange it and got a new club from the pile. Now Joey has two cards, one heart and one club. Grey got the spade card and started a new round.

The second round to the fifth round, they kept picking club from the pile and Joey did nothing. Grey got 4 clubs.

The sixth round, they picked one diamond. Joey used his club to exchange it. Now Joey owns two cards, one heart and one diamond.

The seventh round, they picked one heart, it can be proved that they will stop here.

For the second example, obviously, Grey can get infinity number of cards, so the output is *INF*.

F Paulliant loves photography. Today, he arrived in Harbin to take photos at various landmarks. For simplicity, Harbin can be represented as a **undirected graph** with n nodes and m edges, where each node represents a different landmark and each edge represents a road connecting these landmarks. Paulliant will choose a landmark to start his photography tour. Once he finishes taking photos at one landmark, he will travel along the edges to his next preferred landmark to continue his tour. Due to time constraints, he can visit at most 5 distinct landmarks. Naturally, Paulliant does not want to repeat his visits, so these landmarks must be different. Before the tour,

300分

Paulliant has assigned a "happiness value" to each landmark, where the i -th landmark has a happiness value a_i . Paulliant wants the total happiness value from all the landmarks he visits to be maximized. Determine the maximum total happiness value Paulliant can achieve.

Input

The first line contains two space-separated integers n ($1 \leq n \leq 5000$) and m ($1 \leq m \leq 5000$), representing the number of landmarks and the number of roads in Harbin, respectively.

The second line contains n space-separated integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^8$), representing the happiness value of each landmark.

The following m lines, each contains two space-separated integers u_i and v_i ($1 \leq u, v \leq n$), indicating a road connecting the landmarks u_i and v_i . It is guaranteed that each road is unique.

Output

Output a single integer representing the maximum total happiness value Paulliant can achieve from his photography tour.

Example1

input

```
4 5
5 6 3 4
1 2
1 3
2 3
2 4
3 4
```

output

```
18
```

Example2

input

```
7 10
6 7 8 6 1 7 10
1 2
1 5
2 3
2 5
3 5
3 6
```

3 7
4 5
4 6
5 6

output

34

G Khada_Jhin is learning theoretical computer science.

300分

Today, Jwong asked Khada_Jhin a question: for a string of length n consisting of 0 and 1, how many fundamentally different arrangements of grey code are there?

A permutation of length n is an array $p = p_0p_1p_2\dots p_{n-1}$ consisting of n distinct integers from 0 to $n - 1$ in any order. A circular permutation is treated as a special permutation whose p_0 is always 0.

An arrangement of grey code is a circular permutation $p = p_0p_1p_2\dots p_{2^n-1}$, such that all p_i and p_{i+1} are just one digit different in binary. In particular, p_{2^n-1} and p_0 also need to satisfy just one digit difference in binary.

But this question is so hard for Khada_Jhin, so Jwong simplifies the problem, now Khada_Jhin must compute for a string of length n consisting of 0, 1, how many fundamentally different arrangements of grey-like code?

We also define an arrangement of grey-like code as a circular permutation $p = p_0p_1p_2\dots p_{2^n-1}$, such that for all p_i and p_{i+1} , the last $n - 1$ bit of p_i must be the same as the first $n - 1$ bit of p_{i+1} in binary. In particular, p_{2^n-1} and p_0 also need to satisfy the last $n - 1$ bit of p_{2^n-1} must be the same as the first $n - 1$ bit of p_0 in binary.

For example, when $n = 2$, the answer is equal to 1. Just one legal answer is 00, 01, 11, 10 in binary.

Now it becomes an easy question for Khada_Jhin, but he wants to check if he made a mistake, so please tell him what the answer is modulo 998244353.

Input

Just input one integer n , satisfying $1 \leq n \leq 10^9$.

Output

Just output one integer, which is the answer modulo 998244353.

Example1

input

2

output

1

Example2

input

114514

output

518172623

H Snow wants to buy some goods to celebrate the Spring Festival.

300分

He found that there were n kinds of goods in the store, and each kind of goods had several colors on it. (It means two goods have the same colors if they are of the same kind.) Each color will be represented as a number between 1 and m .

Snow wants to buy k goods so that the set of colors on those goods is exactly what he wants. Snow wants to know how many different plans he can buy. (Note that the good in the same kind is also different. For example, the first kind has 3 goods, and you want to choose 2 from the first kind, you have 3 plans, not 1.)

More formally, Snow will make q queries, each providing you with a set of colors. For each query, you must determine the number of plans that involve purchasing exactly k goods, with the color set on those goods equal to the provided set.

Because the answer may be so big that you need to modulo it by 998244353.

Input

The first line contains three integers m , n , and k ($1 \leq k \leq 100000, 1 \leq n \leq 200000, 1 \leq m \leq 16$).

Then n lines follow. The i -th of them contains two integers w_i and c_i , w_i meaning the number of colors in this kind of goods, c_i represents the number of goods in kind i , followed by w_i distinct integers $col_{i,1}, col_{i,2}, \dots, col_{i,w_i}$, where $col_{i,j}$ denotes the j -th color on i -th kind ($1 \leq w_i, col_{i,j} \leq m, 1 \leq c_i \leq 10^9$).

Then one line contains an integer q , denotes the number of queries ($1 \leq q \leq$

100000).

Then q lines follow. The i -th of them contains an integer $|S|$ meaning the size of color set, and followed by S distinct integers $S_1, S_2, \dots, S_{|S|}$, which denotes colors in this set ($1 \leq |S|, S_i \leq m$).

Output

For all queries, print an integer which means the number of plans modulo 998244353.

Example1

input

```
4 8 2
3 1 1 2 3
2 1 2 4
1 1 4
3 1 1 2 4
2 1 1 3
4 1 1 2 3 4
1 1 3
2 1 1 3
2
4 1 2 3 4
3 2 3 4
```

output

```
15
1
```

Example2

input

```
4 1 1
1 3 1
1
1 1
```

output

```
3
```

Note

For the first example, every kind only has one good.

For the first query, you can choose plans as follows: $(1, 2), (1, 3), (1, 4), (1, 6), (2, 5),$

$(2, 6), (2, 8), (3, 6), (4, 5), (4, 6), (4, 7), (4, 8), (5, 6), (6, 7), (6, 8)$. (Each plan is expressed in the form (i, j) , indicating the selection of kind i and kind j).

For the second query, your only plan is $(2, 7)$.

For the second example, you want to choose 1 good in the first kind, and the first kind has 3 goods, so you have $C_3^1 = 3$ plans.

I As you can see, this is an easy problem.

300分

You are given an integer x , and you only need to output how many "1" in its binary representation.

For example, the integer 5 in binary representation is "101", which has 2 "1", so you should output "2".

Input

Only one integer x , and $1 \leq x \leq 10^9$.

Output

Only one integer, the number of "1" in the binary representation of x .

Example1

input

5

output

2

Example2

input

1024

output

1

J In a prosperous country, Kingsnow decided to engage in trade.

300分

The country consists of $n * m$ cities. Each city is represented by an integer pair

(x, y) , where $1 \leq x \leq n$ and $1 \leq y \leq m$, indicating its position in a grid. In city (x, y) , the item's price is denoted by $a[x][y]$, and the travel expense to reach this city is denoted by $b[x][y]$.

Before embarking on his journey, Kingsnow needs you to plan a route for him. The route must meet the following criteria:

- The route must start at the city located at the first row and first column, i.e., $(1, 1)$.
- The route must end at a city located either at the last row ($x = n$) or the last column ($y = m$).
- Kingsnow can only move from a city (x_i, y_i) to either $(x_i + 1, y_i)$ or $(x_i, y_i + 1)$. Thus, for each step i in the route (except for the last step in the route), (x_{i+1}, y_{i+1}) must be chosen such that it is either directly below or directly to the right of (x_i, y_i) .

Once on the route, Kingsnow will purchase an item in the first city of the route $(1, 1)$. He will then arbitrarily select another city on the route to sell this item. Additionally, he will pay travel expenses for every city visited between the city where he starts the journey and the city where he sells the item.

That is to say, for any given route $(x_1, y_1), (x_2, y_2), \dots, (x_k, y_k)$, Kingsnow will arbitrarily select an integer t from the interval $[2, k]$. The profit he makes from selling the item in city (x_t, y_t) will be calculated as:

$$\text{Profit} = a[x_t][y_t] - a[1][1] - \sum_{i=1}^t b[x_i][y_i]$$

Kingsnow seeks a route such that no matter which city along the route he chooses to sell his item, he will always make a nonnegative profit.

Input

The first line contains two integers n and m ($1 \leq n, m \leq 1000$)--- the number of lines and rows. Ensure that n and m do not equal 1 at the same time.

For the next n lines, each line contains m integers in range $[1, 10^9]$ denoting the price of the item in each city. The y -th integer in the x -th line is $a[x][y]$.

For the next n lines, each line contains m integers in range $[1, 10^9]$ denoting the travel expense in each city. The y -th integer in the x -th line is $b[x][y]$.

Output

If there is at least one route that meets the requirements output "YES", otherwise output "NO".

Example1

input


```
4 4
1 3 4 2
1 0 5 3
5 2 6 1
3 2 7 3
1 1 1 1
1 1 1 1
1 1 1 1
1 1 1 1
```

output

YES

Example2

input

```
4 4
1 5 5 4
5 5 5 4
5 5 5 4
4 4 4 4
1 1 1 1
1 1 1 1
1 1 1 1
1 1 1 1
```

output

NO

K The 24 puzzle is a classical arithmetical puzzle in which the objective is to find a way to manipulate four integers so that the end result is 24.

300分

Nerifish is fond of this kind of puzzle, so he designed a similar puzzle for himself.

Nerifish has four Poker cards, the numbers on each card are a, b, c, d ($1 \leq a, b, c, d \leq 13$). He can arrange the cards in any order and connect them with plus, minus, and multiplication signs. Please note that brackets are **NOT** allowed to be used, the final expression should contain 4 integers and 3 operators.

Nerifish wants to know how many results he can get.

Input

The first line of the input contains four integers a, b, c and d ($1 \leq a, b, c, d \leq 13$), indicating the numbers on each card.

Output

Output one line containing one integer indicating the number of results Nerifish can get.

Example

input

3 1 2 2

output

21

Note

When the final expression is

- " $1 - 3 * 2 * 2$ " can get -11
- " $1 - 3 * 2 - 2$ " can get -7
- " $1 - 2 * 2 - 3$ " can get -6
- " $2 - 1 - 2 * 3$ " can get -5
- " $2 - 3 - 2 - 1$ " can get -4
- " $2 - 3 - 2 * 1$ " can get -3
- " $3 - 2 - 2 - 1$ " can get -2
- " $1 * 3 - 2 - 2$ " can get -1
- " $3 - 2 - 2 + 1$ " can get 0
- " $1 * 2 * 2 - 3$ " can get 1
- " $2 * 2 - 3 + 1$ " can get 2
- " $3 * 2 - 2 - 1$ " can get 3
- " $3 * 2 - 2 * 1$ " can get 4
- " $3 * 2 - 2 + 1$ " can get 5
- " $3 + 2 * 2 - 1$ " can get 6
- " $3 * 2 + 2 - 1$ " can get 7
- " $3 * 2 + 2 * 1$ " can get 8
- " $3 * 2 + 2 + 1$ " can get 9
- " $3 * 2 * 2 - 1$ " can get 11
- " $3 * 2 * 2 * 1$ " can get 12
- " $3 * 2 * 2 + 1$ " can get 13

There are a total of 21 different results here.

" $-3 * 2 * 2 - 1$ " can get -13, but this expression is illegal(it does not meet the requirements of "3 operators").

" $3 + 2 + 2 + 1$ " can get 8, but there is already an expression " $3*2+1$ " which can also get 8.

" $3 * 2 * (2 + 1)$ " can get 18, but this expression is illegal(it does not meet the requirements of "no brackets").

- L The better a person plays badminton, the higher the requirements for badminton rackets will be. Joey is looking at a badminton racket now and evaluating who it is suitable for.

279分

The strings on a badminton racket can be described as a graph with n nodes and m directed edges. Each node i has an elasticity value a_i and toughness value b_i .

A badminton player's ability can be represented by an integer A . When he uses the badminton racket, all nodes i with $a_i \leq A$ will be considered activated, and other nodes will be considered inactive. Let $s_i = 1$ represent node i is activated, and $s_i = 0$ represent node i is inactive.

When a badminton racket is used, each node i has a pressure value p_i . The definition of p_i is as follows:

$$p_i = \max_{\substack{\pi=(v_1,v_2,\dots,v_q) \\ v_q=i}} \left(\sum_{\exists t \in [1,q]: v_t=k} s_k a_k \right)$$

π is a path in the directed graph, $v_1, v_2, \dots, v_q$ are q points in sequence on the path(the same point or edge can be passed through repeatedly). Therefore, this definition means that p_i is equal to the maximum sum of $s_k a_k$ of all nodes k among all paths ending with i .

If each node i satisfies $p_i \leq b_i$, then this badminton racket is suitable for this badminton player.

Joey wants to know what the maximum A a badminton player can have. (note that this badminton racket should be suitable for this player)

Input

The first line contains two integers n and m ($1 \leq n, m \leq 10^5$), the number of nodes and edges.

The second line contains n integers, and the i -th integer a_i ($1 \leq a_i \leq 10^9$) represents the elasticity value of node i .

The third line contains n integers, and the i -th integer b_i ($1 \leq b_i \leq 10^9$) represents the toughness value of node i .

For the next m lines, each line contains 2 integers u, v ($1 \leq u, v \leq n$), which represents a directed edge from u to v .

Output

Output the largest A the badminton player can have, if A can be larger than 10^9 then output 10^9 .

Example

input

```
5 10
10 1 3 5 6
9 9 5 10 14
1 5
4 2
4 2
2 2
5 1
5 1
3 4
4 1
1 1
3 4
```

output

```
5
```
